arXiv:2408.14906v1 [cs.CL] 27 Aug 2024

Writing in the Margins: Better Inference Pattern for Long Context Retrieval

Melisa Russak **Umar Jamil** **Christopher Bryant** **Kiran Kamble** **Axel Magnuson** **Mateusz Russak**

Waseem AlShikh

Writer, Inc.
{melisa,...,waseem}@writer.com

Abstract

In this paper, we introduce Writing in the Margins (WiM), a new inference pattern for Large Language Models designed to optimize the handling of long input sequences in retrieval-oriented tasks. This approach leverages the chunked prefill of the key-value cache to perform segment-wise inference, which enables efficient processing of extensive contexts along with the generation and classification of intermediate information ("margins") that guide the model towards specific tasks. This method increases computational overhead marginally while significantly enhancing the performance of off-the-shelf models without the need for fine-tuning. Specifically, we observe that WiM provides an average enhancement of 7.5% in accuracy for reasoning skills (HotpotQA, MultiHop-RAG) and more than a 30.0% increase in the F1-score for aggregation tasks (CWE). Additionally, we show how the proposed pattern fits into an interactive retrieval design that provides end-users with ongoing updates about the progress of context processing, and pinpoints the integration of relevant information into the final response. We release our implementation of WiM using Hugging Face Transformers library at https://github.com/writer/writing-in-the-margins.

1 Introduction

The performance of Large Language Models (LLMs) tends to deteriorate when processing extensive inputs, a limitation linked directly to their fixed context window and attention mechanisms [23, 24]. In particular, LLMs struggle with tasks involving long contexts, especially when the relevant information is embedded in larger volumes of text [38, 28]. Recent research thus highlights the importance of improving model capabilities to handle more extensive datasets without losing accuracy or requiring exponential increases in computational resources.

There have been various attempts to extend the usable context window of LLMs, such as sparse attention [31, 40, 25], length extrapolation [5, 30, 27], and context compression [42, 44]. Concurrently, the field has witnessed the rise of sophisticated prompting strategies like Chain of Thought (CoT) and related structured reasoning methods [33, 35, 39]. These approaches have significantly enhanced LLMs' ability to tackle complex tasks by systematically guiding the reasoning process through predefined structural patterns.

Our work bridges the gap between efficient transformers architecture research and development of new prompting strategies. Specifically, we identify a novel key-value (KV) cache aware reasoning pattern for existing off-the-shelf long context window LLMs in scenarios typical of retrieval-oriented tasks, where the context is substantial and the instructional prompt is comparatively short. We begin by recognizing that long-context prompts are commonly prefilled in the KV cache segment-wise in a process known as chunked prefill. From this insight, we introduce an inference pattern called Writing in the Margins (WiM), which concurrently generates query-based extractive summaries at each step of the prefill that are subsequently reintegrated at the end of the computation. We term these intermediate outputs "margins", drawing inspiration from the practice of making margin notes for improved comprehension of long contexts in human reading. Using methodologies similar to "scratchpad" techniques, which meticulously record step-by-step calculations, we incorporate margin notes into the final segment predictions. We show that this technique, which adds only minimal additional computation, significantly enhances long context comprehension. The WiM pattern can also provide end-users with real-time insights into computational progress

Preprint. Under review.

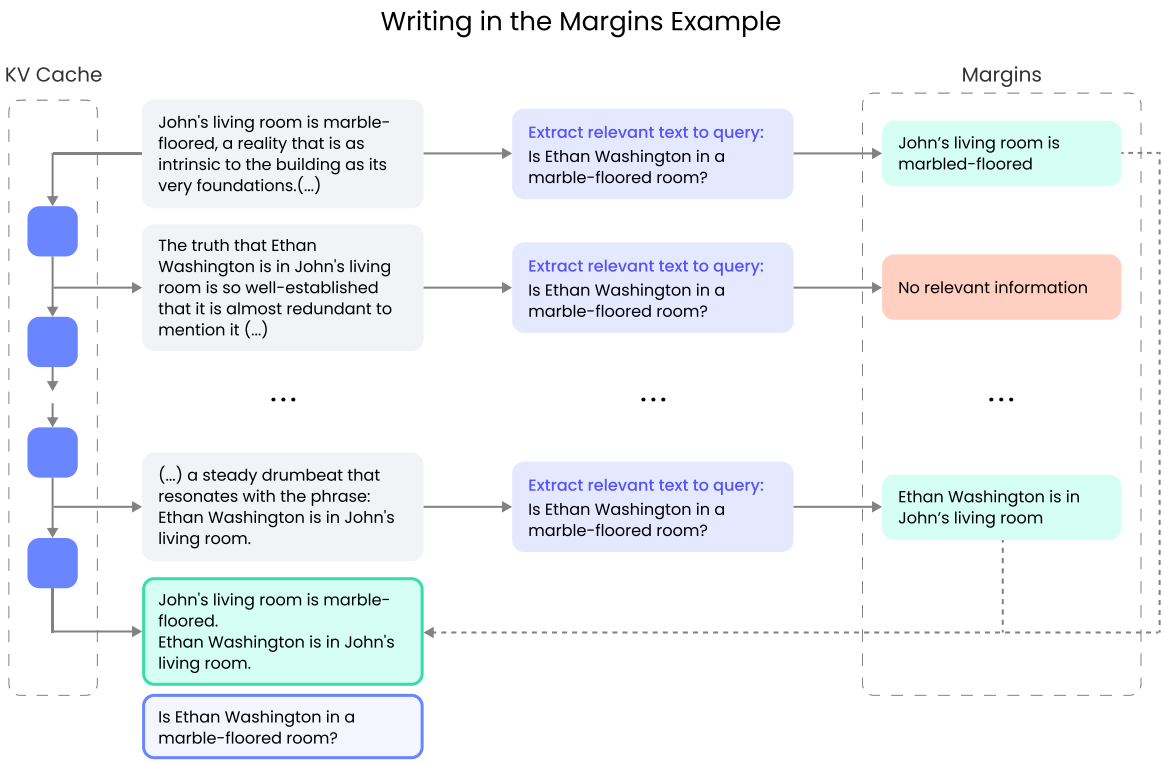

Figure 1: **Writing in the Margins inference pattern.** Prefilling KV cache by segments allows to both process the context segment by segment and generate intermediate extractive summaries which can improve the final prediction.

through streamed margin notes, which ultimately help make AI decisions more transparent and explainable. This can enable users to (1) pinpoint the location of essential information and (2) reduce computational load by exiting early if the provided information satisfactorily addresses the query.

In Figure 1, we provide an illustrative example of WiM inference, which we encourage readers to reference as a practical demonstration to complement the formal algorithm description that will be presented in the following sections.

Our main contributions are as follows:

a) We introduce a new inference pattern, Writing in the Margins (WiM), which achieves better performance on long-context window tasks with a relatively minor increase in computational cost.

b) We demonstrate the application of WiM within an interactive long context retrieval setup, effectively increasing the transparency of the process and reducing the first response latency.

c) We provide an implementation of this inference pattern using the Hugging Face Transformers library.

2 Writing in the Margins

Chunked Prefill Typically, the process of inference for generative LLMs consists of two principal phases: the prefill phase and the decoding phase. When an LLM is requested to prefill a substantial prompt—in the range of hundreds of thousands of tokens—it is common practice to prefill the KV cache in chunks [20]. This method is known as chunked prefill and is supported by many inference frameworks, including vLLM [47].

Chunked prefill divides the prompt into fixed-size chunks to populate the KV cache at each layer of the Transformer model [32]. The rationale for chunked prefill is to reduce overall memory usage, as the quadratic memory complexity of the attention mechanism during prefilling can be prohibitive for larger prompts. By splitting a prompt of length L into N chunks, each of size K, where $N = L/K$, the overall memory complexity of prefilling is reduced from $O(L^2)$ to $O(LK)$. The attention mask must be adjusted to allow each new chunk to attend to all tokens in the previous chunks while maintaining the causal structure only for the new chunk, as illustrated in Figure 2.

Our work exploits the chunked prefill mechanism to generate intermediate "margins" that can then be appended to the prompt to better guide the model toward performing a specific task.

	K0	K1	K2	K3
Q0	1	0	0	0
Q1	1	1	0	0
Q2	1	1	1	0
Q3	1	1	1	1

	K0	K1	K2	K3	K4	K5	K6	K7
Q4	1	1	1	1	1	0	0	0
Q5	1	1	1	1	1	1	0	0
Q6	1	1	1	1	1	1	1	0
Q7	1	1	1	1	1	1	1	1

Figure 2: **Chunked Prefill.** Example of how the attention mask is set across different chunks during prefill iterations (first chunk at the top, second chunk at the bottom). Each new chunk needs to retain causality while attending to all previous chunks. Chunked prefill is mathematically equivalent to prefill without chunking.

Writing in the Margins Consider a prompt P, composed of a context C, and an instruction I.

$$\text{P} = \text{C} + \text{I} \tag{1}$$

Prefilling a decoder-only transformer model T directly with the entire prompt $T(P)$ is computationally inefficient when the prompt is long. Moreover, as shown in [24], processing the entire prompt in one go can lead to mid-sequence forgetting.

To make this process more efficient, we implement the prefill technique described in the previous paragraph, where the context C is divided into N segments.

$$\text{C} = c_1 + c_2 + \ldots + c_N \tag{2}$$

For the first segment, the model T operates on chunk c_1, resulting in output that includes past key values pkv_1. The model continues onto the second segment with the pkv_1 cached, i.e., $T(\text{pkv}_1, c_2)$, effectively emulating the scenario of processing $T(c_1 + c_2)$ in one step. As the procedure progresses, each sequential chunk, c_k, is processed with prefilled past key values, noted as $T(\text{pkv}_{[1..k-1]}, c_k)$, mimicking an uninterrupted run of T on C.

The Writing in the Margins (WiM) strategy addresses potential mid-sequence forgetting issues by appending an extractive instruction I_A to each chunk, enhancing chunk-specific outputs. It transforms each step into $T(\text{pkv}_{[1..k-1]}, c_k + I_A)$, where the instruction I_A is embedded alongside each context chunk, then dropped from the KV cache before the next chunk prefilling. The instruction I_A is closely related to I - the model is asked to copy over all relevant to I information.

Intermediate outputs from each chunk are referred to as margin notes M_i, cumulatively forming N notes, described as $M = M_{[1..N]}$. Unhelpful notes, perhaps irrelevant to the instruction, are discarded, enhancing the final contextual construct to C + M + I, positioned advantageously towards the end to minimize mid-sequence memory loss. Intuitively, the model is allowed to use relevant intermediate predictions while answering the final query.

To summarize, we modify the chunked prefill algorithm by adding extra decoding steps (green in Table 1). Most of these steps can be efficiently batched with the original prefill steps. The query-relevant information extracted from these steps is then added at the end of the context but before the instruction.

step	Chunked Prefill	WiM	keep
1	$T(\varnothing, c_1)^{\dagger}$	$T(\varnothing, c_1)^{\dagger}$	$\text{pkv}_{[1]}$
2	$T(\text{pkv}_{[1]}, c_2)^{\dagger}$	$T(\text{pkv}_{[1]}, c_2)^{\dagger}$ $T(\text{pkv}_{[1]}, I_A)^{\dagger\ddagger}$	$\text{pkv}_{[1..2]}$ M_1
⋮	⋮	⋮	⋮
N	$T(\text{pkv}_{[1..N-1]}, c_N)^{\dagger}$	$T(\text{pkv}_{[1..N-1]}, c_N)^{\dagger}$ $T(\text{pkv}_{[1..N-1]}, I_A)^{\dagger\ddagger}$	$\text{pkv}_{[1..N]}$ M_{N-1}
N + 1		$T(\text{pkv}_{[1..N]}, I_A)^{\dagger\ddagger}$	M_N
N + 2	$T(\text{pkv}_{[1..N]}, I)^{\dagger\ddagger}$	$T(\text{pkv}_{[1..N]}, M_{[1..N]} + I)^{\dagger\ddagger}$	

Table 1: **Batching Chunked Prefill Steps with WiM margin generation.** The inference for generative LLMs consists of two principal phases: the prefill phase (†) and the decoding phase (‡). WiM algorithm adds extra decoding steps that mostly can be batched with chunked prefill steps. We keep margin notes M_i produced in extra steps (green) as plain text. We then prefill the model T with all relevant notes $M_{[1..N]}$ before the final instruction I.

Algorithms 1 and 2 present pseudocodes of inference with chunked-prefill and WiM respectively.

Generating and classifying margins It is possible to use the same instance of the model for both generating the margins and classifying them, without affecting the prefilled KV cache. Having generated the margins, it is possible to inference the model with a classification prompt without using the KV cache (`past_key_value` in Algorithm 2) generated by the prefilling operation. This way, the model will act as it had never been prefilled. Having classified the margins, it is possible to reuse the previously prefilled KV cache to append the classified margins and then generate the final output. The overhead of classifying a single margin, in terms of memory, is just the KV cache

size of a single margin and the classification prompt, which is negligible compared to the prefilled long-context prompt. It is also possible to overlap the generation of margins with their classification using the same model instance and the same request in the batch (see Appendix A for more details).

Algorithm 1: Inference with Chunked Prefill

```
Input  : system_message (string)
         context (string)
         instruction (string)
         llm (object)
Output : output (string)
context ← system_message + context;
segments ← split(context);
past_key_value ← [];
for segment ∈ segments do
    // add the segment to the KV cache
    prefill(llm, past_key_value, segment);
end
output ← generate(llm, past_key_value, instruction);
return output
```

Algorithm 2: Writing in the Margins

```
Input  : system_message (string)
         context (string)
         instruction (string)
         extractive_summary_prompt (string)
         classification_prompt (string)
         llm (object)
Output : output (string)
context ← system_message + context;
segments ← split(context);
past_key_value ← [];
positive_margins ← [];
for segment ∈ segments do
    // add the segment to the KV cache
    prefill(llm, past_key_value, segment);
    // generate using the content of the KV cache
       and then discard any
    // tokens added to the KV cache by the prompt
       and the generated tokens
    margin ← generate(llm, past_key_value,
      extractive_summary_prompt);
    classification_input ← format(classification_prompt,
      margin, instruction);
end
// do not use any past KV cache to classify
classification_result ← generate(llm, NULL,
  classification_input);
if classification_result = true then
    append(positive_margins, margin)
end
all_positive_margins ← concatenate(positive_margins);
prefill(llm, past_key_value, all_positive_margins);
output ← generate(llm, past_key_value, instruction);
return output
```

[1]RULER code: https://github.com/hsiehjackson/RULER

3 Experimental Setup

3.1 Datasets

Following the RULER [43] task categories, we measure the performance of an inference pattern on three types of skills: **(I) Multi-Hop Reasoning**, **(II) Needle Retrieval/ Single-Hop Reasoning**, and **(III) Aggregation**. Table 2 presents the curated long context datasets used to benchmark all LLMs:

skill type	benchmark name	context length (tokens)	# examples
I	MultiHop-RAG ♣ [46]	13-32k	100
I	HotpotQA † [4]	16k/ 32k /64k	100/ 100/ 100
II	SQuAD † [3]	16k/ 32k/ 64k	100/ 100/ 100
III	CWE † [43]	64k	100

Table 2: **Datasets** We have curated four datasets to evaluate long context window LLMs. Each set consists of 100 examples, generated either using the RULER code (†) or by subsampling the longest examples from the original benchmark data (♣).

In the following paragraph, we briefly introduce the benchmarks used in each category, describe our curating rationale, and hypothesize the impact of WiM on each skill set.

I **Multi-Hop QA** The task aims to check the behavior of tracing entities with multi-hop connections based on the HotPotQA and MultiHop-RAG benchmarks [46, 4]. We used the RULER codebase[1] to generate a subset of 100 examples based on HotPotQA - a multi-hop queries sourced from Wikipedia articles. Following RULER, we simulated long context retrieval scenarios by generating examples in three length variants: $16k$, $32k$, $64k$. We also selected the 100 longest examples in the range of $13k$-$33k$ tokens from MultiHop-RAG - a large collection of multi-hop queries based on English news articles.

Hypothesis Transformers are known for not being able to emulate a for loop [36]. In WiM we simulate going through the context twice which can improve the performance by aggregating all interconnected facts in one place at the end of the document.

II **Needle Retrieval/ Single-Hop Reasoning** In the context of a long context window, the Needle Retrieval and Single-Hop QA task can be jointly seen as a kind of filter benchmark, where the task is to filter irrelevant content and either copy or transform the relevant information. We used the RULER code to generate examples based on SQuAD [3] in three context length variants: $16k$, $32k$ and $64k$, collecting 100 datapoints in each variant.

Hypothesis The reduction using the WiM inference pattern is in fact a reverse engineering of how filter-type benchmarks are created. The model is asked to filter out injected distractions and copy relevant parts into the margin in each segment-wise prediction step.

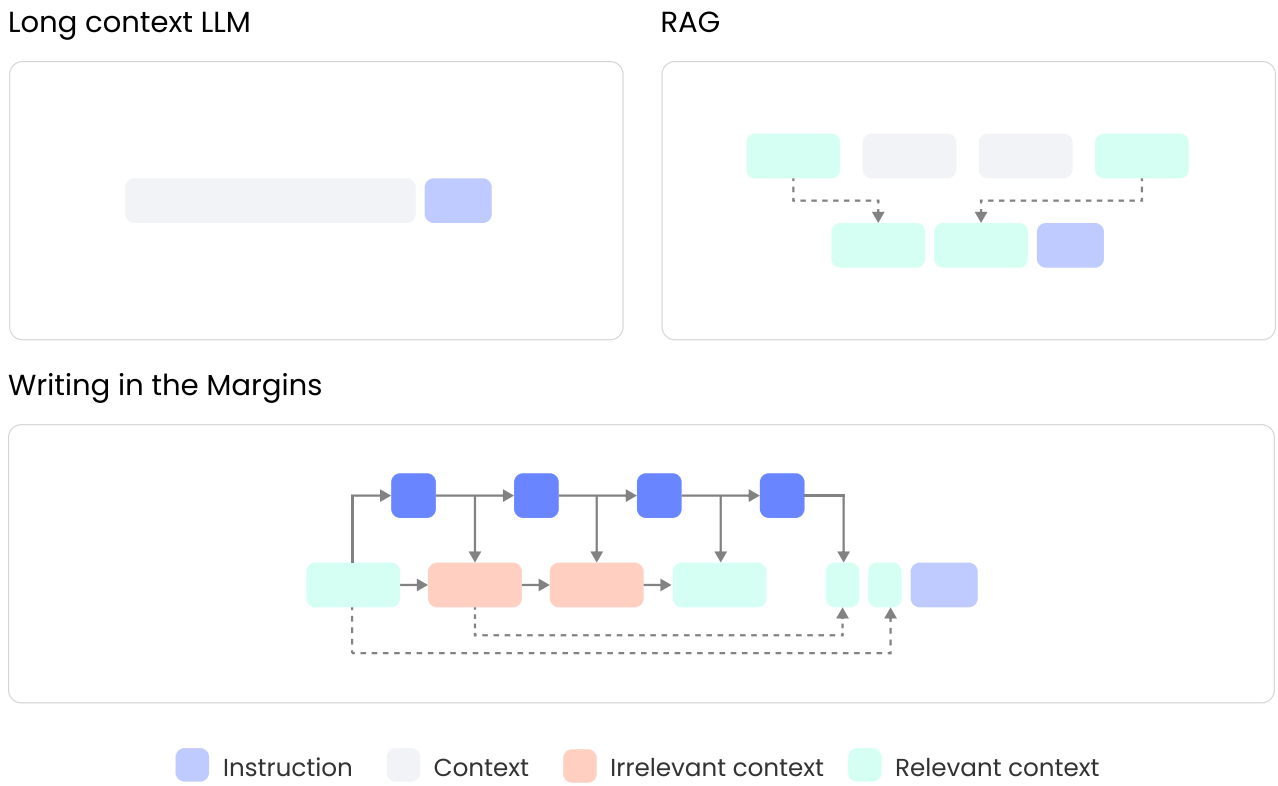

Figure 3: **Design Comparison.** Three inference designs for managing long context windows: (Top Left) Long Context LLM (LLM): This design feeds all context, without segmentation, directly to the model. (Top Right) Retrieval-Augmented Generation (RAG): Segments are selected based on a retrieval method (e.g., cosine similarity between vector representations of the query and the segment). All selected segments, along with the task instruction, are then concatenated and fed to a model. (Bottom) Writing in the Margins (WiM): The context is divided and processed segment by segment. At each step, the model is prompted to produce auxiliary information from each segment. This information is then classified and, if deemed positive, it is incorporated into the final step before the task description.

III **Aggregation** This type of task measures a model's ability to aggregate relevant information that spans long-range context. We use the Common Words Extraction (CWE) benchmark [43] where words are sampled from discrete distributions, with the number of common words fixed and increasing with the sequence length number of uncommon words. Following RULER, we generated 100 examples of average length $64k$ tokens. We scaled original frequencies of common and uncommon words to match the extended length of samples: common words appear 500 times in the sample, while uncommon words do not appear more than 50 times. We modified the task instruction to include the number of word occurrences in text to support aggregation over segments.

Hypothesis The performance of WiM in reduction tasks will be related to the concept of hierarchical reduction. We can think of a summarization task as being a summary of segment summaries, or any aggregate task as being a result over cumulative aggregation over segments.

3.2 Long Context Window LLMs

We selected seven off-the-shelf models that officially support context windows up to $128k$ tokens:

- Phi-3-small-128k-instruct[2]
- Qwen2-7B-Instruct[3]
- Meta-Llama-3.1-8B-Instruct[4]
- Phi-3-medium-128k-Instruct[2]

[2]Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. [37]
[3]Qwen2 Technical Report. [48]
[4]The Llama 3 Herd of Models. [41]
[5]Writer's proprietary model.

- Palmyra-4-Chat-128K[5]
- Meta-Llama-3.1-70B-Instruct[4]
- Qwen2-72B-Instruct[3]

In all experiments, we used half precision models with identical sampling parameters — specifically, a temperature setting of 0.0 and $2k$ maximum new tokens. We used 0-shot prompts for all benchmarks. In MultiHop-RAG, HotPotQA and SQuAD experiments, we applied the same model-independent preposessing step: we used nltk [2] to split the context into sentences, then grouped them in segments no longer than 4096 tokens. This resulted in 4 – 16 margin notes per datapoint. In CWE, where the datapoints contain only numbered words, we exchanged nltk for naive words split by space and used 8192 segment length, which gave on average 8 margins per sample. We chose to count tokens using GPT-4 tiktoken tokenizer[6] since this choice does not favour any of the evaluated models' tokenizers.

In each case, we measured the relative differences of WiM pattern scores with respect to the following two baselines:

- **Long Context LLM (LLM)** - all context without segmentation is fed to the LLM.
- **Retrieval Augmented Generation (RAG)** - segments are selected based on a retriever (ex. cosine similarity between vector representations of the query and the segment), then all selected segments and the task instruction are concatenated and fed to an LLM.

All three inference patterns, including WiM, are presented in Figure 3.

In order to make the results more comparable, we replaced the retriever in RAG with the classifier used in WiM. We expect the RAG results to be lower in the real RAG systems (especially for longer segment lengths), as vectorization is a form of lossy compression.

3.3 Prompting

For all benchmarks, we respected their original formulation. In all cases, the prompt strategy for the Long Context LLM baseline could be expressed as:

```
{system_message}
‘‘‘text
{context}
‘‘‘
{instruction}
{query}
```

Where `system_message` and `instruction` were usually the task instructions split into two parts and appended before and after the main context respectively.

In the RAG approach, we used the original prompt but replaced `context` with all relevant segments concatenated by a newline sign.

[6] https://github.com/openai/tiktoken

In WiM inference, all constructed prompts shared the common prefix:

```
{system_message}
‘‘‘text
{context}
‘‘‘
```

This was necessary for the efficient reuse of the KV cache. To ensure that predictions were comparable, we manually identified a promising prompt for the margin generation and final prediction steps for all evaluated models.

3.3.1 Margin generation

For each intermediate context $\texttt{context}_i = \Sigma_1^i c_i$ and instruction I, we used the following extractive summary prompt I_A to generate a margin note M_i:

```
I_A = """
{system_message}
‘‘‘text
{context_i}
‘‘‘
Copy over all context relevant to the query: {query}
Provide the answer in the format: <YES/NO>#<Relevant
    ↪  context>.
Here are rules:
- If you don’t know how to answer the query - start
    ↪ your answer with NO#
- If the text is not related to the query - start
    ↪ your answer with NO#
- If you can extract relevant information - start
    ↪ your answer with YES#
- If the text does not mention the person by name -
    ↪ start your answer with NO#
Example answers:
- YES#Western philosophy originated in Ancient
    ↪ Greece in the 6th century BCE with the pre-
    ↪ Socratics.
- NO#No relevant context.
"""
```

In our experiments, the margin generation step was combined with the classification step; the first token generated was a class label. We conditioned the generation of a margin based on the first token; i.e., we continued the generation only if the first token was `YES`. Additionally, the prompt included an explanation designed to enforce specific formatting and to prevent the model from inserting comments before delivering its judgment.

In Appendix A, we explore the possibility of decoupling margin generation and classification prompts while using the same instance of the model.

3.3.2 Final WiM prompt with accumulated margins

To distinguish the content of the margins from the original context, and to maintain the document's logic and structure, we explicitly named the writing-in-the-margins strategy in our last step, while aggregating all relevant margin notes.

We used two variants of the prompt, depending on the number of retrieved margins.

Single margin

```
{system_message}
‘‘‘text
{context}
‘‘‘
I asked my assistant to read and analyse the above
    ↪ content page by page to help you complete
    ↪ this task. This is a margin note left on the
    ↪ last page:
‘‘‘text
QUERY: {query}
ANSWER: {M_i}
‘‘‘
Read again the note(s) and the provided content,
    ↪ take a deep breath and answer the query.
{instruction}
{query}
```

Multiple margins

```
{system_message}
‘‘‘text
{context}
‘‘‘
I asked my assistant to read and analyse the above
    ↪ content page by page to help you complete
    ↪ this task. Those are margin notes left on
    ↪ each page:
‘‘‘text
Page 0:
QUERY: {query}
ANSWER: {M_i}
Page 1:
QUERY: {query}
ANSWER: {M_j}
...
‘‘‘
Read again the note(s) and the provided content,
    ↪ take a deep breath and answer the query.
{instruction}
{query}
```

We replaced the term “segment” with “page” to more closely replicate the human practice of writing in the margins. In our experiments, there was no relationship between the order of the segments and the page numbers; this is left as an optional implementation detail.

3.4 Evaluation

We used the same 3-shot prompt with GPT-4-turbo [26] and greedy sampling to evaluate models' accuracy in HotpotQA, MultiHop-RAG and SQuAD benchmarks. For the CWE benchmark we adjusted the prompt and examples to calculate precision (P), recall (R) and F1-score. Both prompts are shown in appendix B.

4 Results

Detailed results of the experiment are presented in Table 3.

4.1 Multi-Hop Reasoning

Notably, for almost all evaluated models, WiM improves multi-hop reasoning abilities, on average giving a 7.5% boost with respect to the Long Context LLM inference and 9% with respect to RAG. The most significant performance boost is observed in smaller models — replacing a vanilla Phi-3-small-128k-instruct inference with WiM leads to 19% improvement in MultiHop-RAG benchmark and 12% in HotpotQA.

By looking at different length variants of HotpotQA ($16k$, $32k$, $64k$) we see that all patterns lose accuracy as we add more context (LLM: from 0.65 to 0.54, RAG: from 0.61 to 0.58, WiM: from 0.72 to 0.64). This observation aligns with the notion that extending the context length in models degrades the performance of complex reasoning tasks. However, using WiM allows us to maintain almost the same accuracy for $64k$ as the LLM achieves on $16k$.

4.2 Needle Retrieval and Single-Hop Question Answering

Analysis of the SQuAD benchmark results shows that all scores are distributed across similar values with a slight preference for RAG. It is advisable to reassess this conclusion with different model choices and context lengths; for instance, replacing an LLM with the WiM pattern consistently improves accuracy in SQuAD by $2\% - 17\%$ for Qwen2-7B-Instruct, whereas LLM is a preferred inference pattern for $16k$ context window for 4 out of 7 tested models.

Unsurprisingly, RAG emerges as the most optimal pattern for six out of seven evaluated models when extending the context length to 64k tokens in SQuAD. Indeed, for single-hop reasoning tasks, if the filtering process is successful (here we approximate the retriever by an LLM classifier), the challenge is reduced to a trivial task of retrieving a needle from a context window of 4096 tokens. However, this assumption in the RAG setup is overly optimistic because the LLMs used in our experiment are at least $7B$ in model parameters, and such large models are not typically used as retrievers. In practical scenarios, one might expect the results to be even more favorable for both LLM and WiM compared to RAG.

4.3 Aggregation

The pattern across the data indicates that WiM either matches or substantially boosts the aggregation skills of off-the-shelf models, giving an LLM on average a 30% increase in F1-score for the CWE benchmark, and outperforming RAG by 17%.

We observe that CWE results can be grouped into four classes, which surprisingly tend to align more with the model families than with the model sizes. Models like Meta-Llama-3.1-8B-Instruct and Meta-Llama-3.1-70B-Instruct achieve a remarkably significant boost in F1-score when using WiM across all context lengths, reaching up to 72% compared to the LLM

baseline. Conversely, models like Phi-3-small-128k-instruct and Phi-3-medium-128k-instruct consistently prefer the vanilla LLM inference. Meanwhile, Qwen2-7B-Instruct and Qwen2-72B-Instruct point to WiM as the most optimal pattern, showing a moderate improvement ranging from 22% to 59%. RAG is preferred only by Palmyra-4-Chat-128K, which outperforms the rest by $3\% - 4\%$.

After reviewing all comments provided by GPT-4-turbo during the evaluation of each data point, we observe that models often resort to writing Python code to solve the problem (18.5% of all answers), leading to incorrect or generic answers (resulting in on average 20% drop in F1-score).

5 Ablation Study

5.1 No margins filtering

In this experiment, we excluded the margins classifier from the WiM pipeline, which resulted in all extractive summaries being appended directly to the context.

Table 4 presents the accuracy scores aggregated over three benchmarks: HotpotQA, MultiHop-RAG, SQuAD. Including all margins in the context decreases the accuracy by up to 8% compared to the original WiM pipeline. This effect is analogous to negative instruction manipulation, akin to telling the model to "forget all previous instructions". Ultimately, filtering margins—especially when combined with the margin generation step—not only saves computation by allowing irrelevant margins to be dropped, but also improves the overall performance.

model name	filtered (WiM)	all
Phi-3-small-128k-instruct	**0.58**	0.54
Qwen2-7B-Instruct	**0.65**	0.63
Meta-Llama-3.1-8B-Instruct	**0.70**	**0.70**
Phi-3-medium-128k-instruct	**0.65**	0.64
Palmyra-4-Chat-128K	**0.64**	0.55
Meta-Llama-3.1-70B-Instruct	0.72	**0.73**
Qwen2-72B-Instruct	**0.72**	0.71

Table 4: **Ablation: Filtering Margins** Classification and removal of irrelevant margins yields better results for majority of evaluated models. Results aggregated over HotpotQA, Multihop-RAG and SQuAD benchmarks.

5.2 Replacing the content by margins

One appealing option is to reduce computational demands by entirely eliminating the KV cache in the final step and relying solely on the extracted positive margins. This approach transforms the long context document into a compressed version that is dependent on the query. While we anticipate that retaining the full context might be better at capturing the answer, we have also observed, as noted in the previous section, a decrease in model performance as the input length increases.

Table 5 presents aggregated results for HotpotQA, MultiHop-RAG and SQuAD, demonstrating that incorporating both margins and the complete document consistently maximized the performance for almost all evaluated models, except for Meta-Phi-3-small-128k-instruct. Employing a query-based extractive summary—specifically, using only the content from margins—gave mixed results across all models; e.g., Meta-Llama-3.1-70B-Instruct scores were consistent across all metrics (0.72), while Palmyra-4-Chat-128K scores saw a decrease from 0.64 to 0.53. On the other hand, the model Phi-3-small-128k-instruct experienced an increase from 0.58 to 0.6. We hypothesize that these outcomes might vary depending on the specific task at hand. It is plausible that for tasks such as filtering and improving recall (i.e., when models are fine-tuned for margin generation and classification tasks), using margins could prove beneficial as the filtered-out content would be entirely irrelevant.

model name	only margins	only context	both (WiM)
Phi-3-small-128k-instruct	**0.60**	0.55	0.58
Qwen2-7B-Instruct	0.62	0.56	**0.65**
Meta-Llama-3.1-8B-Instruct	0.68	0.68	**0.70**
Phi-3-medium-128k-instruct	0.57	0.58	**0.65**
Palmyra-4-Chat-128K	0.53	0.63	**0.64**
Meta-Llama-3.1-70B-Instruct	**0.72**	**0.72**	**0.72**
Qwen2-72B-Instruct	**0.72**	0.67	**0.72**

Table 5: **Ablation: Content Compression** Utilizing both margins and the entire context almost always maximizes the score despite the declining performance of the underlying model when faced with increasing input lengths. Results aggregated over HotpotQA, Multihop-RAG and SQuAD benchmarks.

6 Interactive Retrieval

Explainability The design principles behind WiM focus not just on enhancing final benchmark performance, but also on improving the user experience. By presenting intermediate computation steps, WiM renders the decision-making process of LLMs transparent. This clarity in the model's reasoning process aids not only in debugging but also provides insights that are crucial for both end-users and developers, ensuring outputs that are both understandable and reliable.

Latency Handling long documents can degrade user experience due to significant latency, as the model becomes unresponsive during processing, which can take minutes without clear indications of wait time. Our design addresses this by providing relevant information during processing and by segment-wise processing that incorporates a progress bar, thus reducing the initial response latency.

Early exit WiM also offers an "early exit" option, allowing users to stop the computation if they find a satisfactory answer within any of the displayed margins. For example, in single-hop question-answering scenarios, once the answer is found in a particular section, there is no need to process further.

		HotpotQA			MultiHop RAG	SQuAD			CWE			Average
	Context:	**16k**	**32k**	**64k**	**13-32k**	**16k**	**32k**	**64k**	**64k**			**Excl. CWE**
Model	**Pattern**	**Acc.**	**Acc.**	**Acc.**	**Acc.**	**Acc.**	**Acc.**	**Acc.**	**P**	**R**	**F1**	**Acc.**
Phi-3-small-128k-instruct	LLM	0.47	0.55	0.48	0.58	**0.81**	0.75	**0.79**	**0.77**	**0.77**	**0.77**	0.52
	RAG	0.55	0.56	0.50	0.70	**0.81**	**0.78**	**0.79**	0.65	0.64	0.65	**0.58**
	WiM	**0.66**	**0.64**	**0.56**	**0.77**	0.65	0.74	0.64	0.70	0.69	0.69	**0.66**
Qwen2-7B-Instruct	LLM	0.62	0.59	0.39	0.83	0.81	0.71	0.57	0.46	0.46	0.46	0.61
	RAG	0.54	0.55	**0.56**	0.77	**0.87**	**0.84**	**0.86**	0.49	0.49	0.49	0.61
	WiM	**0.69**	**0.66**	**0.56**	**0.92**	0.83	0.80	0.74	**0.69**	**0.67**	**0.68**	**0.71**
Meta-Llama-3.1-8B-Instruct	LLM	0.65	0.64	0.60	0.85	**0.90**	**0.92**	0.87	0.22	0.21	0.22	0.69
	RAG	0.67	0.65	0.59	0.77	0.87	0.91	**0.91**	0.47	0.47	0.47	0.67
	WiM	**0.77**	**0.71**	**0.73**	**0.86**	0.88	0.85	0.82	**0.94**	**0.93**	**0.93**	**0.77**
Phi-3-medium-128k-instruct	LLM	0.57	0.53	0.48	0.80	0.84	0.72	0.70	**0.91**	**0.91**	**0.91**	0.60
	RAG	0.50	0.55	0.51	0.78	**0.86**	**0.82**	**0.83**	**0.91**	**0.91**	**0.91**	0.59
	WiM	**0.63**	**0.67**	**0.57**	**0.93**	0.81	0.80	0.77	0.90	0.90	0.90	**0.70**
Palmyra-4-Chat-128K	LLM	**0.70**	0.60	0.57	0.85	**0.84**	0.76	0.73	0.76	0.77	0.76	0.68
	RAG	0.59	0.54	0.55	0.78	0.74	0.70	0.69	**0.80**	**0.80**	**0.80**	0.62
	WiM	0.69	**0.63**	**0.66**	**0.86**	0.78	**0.77**	**0.74**	0.77	0.77	0.77	**0.71**
Meta-Llama-3.1-70B-Instruct	LLM	**0.80**	0.74	0.70	**0.91**	**0.93**	0.85	0.87	0.37	0.36	0.36	**0.79**
	RAG	0.73	0.72	0.63	0.80	0.90	**0.92**	**0.95**	0.66	0.65	0.66	0.72
	WiM	0.79	**0.76**	**0.71**	0.89	0.90	0.90	0.82	**1.00**	**1.00**	**1.00**	0.79
Qwen2-72B-Instruct	LLM	0.75	0.72	0.57	**0.88**	0.91	0.78	0.76	0.42	0.36	0.39	0.73
	RAG	0.70	0.66	**0.70**	0.80	**0.92**	0.87	**0.91**	0.75	0.75	0.75	0.72
	WiM	**0.80**	**0.79**	**0.70**	**0.88**	0.88	**0.88**	0.87	**0.98**	**0.98**	**0.98**	**0.79**
Average	LLM	0.65	0.62	0.54	0.81	**0.86**	0.78	0.76	0.56	0.55	0.55	0.66
	RAG	0.61	0.60	0.58	0.77	0.85	**0.83**	**0.85**	0.68	0.67	0.68	0.64
	WiM	**0.72**	**0.69**	**0.64**	**0.87**	0.82	0.82	0.77	**0.85**	**0.85**	**0.85**	**0.73**

Table 3: **Main Results** We present results for seven off-the-shelf models and four benchmarks. We used the accuracy metric (Acc.) for all benchmarks except CWE, where we used precision (P), recall (R), and F1-score. Aggregated results (both model-wise and benchmark-wise) show that WiM maximizes performance in multi-hop reasoning and summarization-like tasks (HoppotQA, Multihop-RAG, and CWE). For single-hop reasoning (SQuAD), the results vary across models.

Human in the Loop Users have the ability to improve the decision-making process by adding labels to the margins displayed in WiM. In this design, the final answer considers both the full context and the user-labeled margins. Users can evaluate and label the streamed margins (e.g., with a thumbs up or down), and these inputs could be reintegrated into the final decision-making step. The proposed design, including this feedback loop, is illustrated in Figure 4.

7 Conclusion

In this paper, we have introduced a new inference pattern called Writing in the Margins (WiM), which leverages chunked prefill to add only a marginal computational cost, emulating the human behavior of making notes in the margins. We demonstrated that this inference pattern significantly boosts the performance of off-the-shelf models across various long-context, retrieval-oriented tasks, including multi-hop reasoning (by 7.5% in HotpotQA, MultiHop-RAG), and aggregation (by 30.0% in CWE). Remarkably, this method does not require finetuning and is compatible with any transformer model.

Additionally, our approach improves end-user experience by increasing transparency in context processing. By streaming "margins" that contribute to the final model predictions, our design offers the potential for early engagement. Unlike conventional long-context inference methods, WiM allows the first relevant margin to be streamed immediately after processing the corresponding segment. This not only improves latency but also reduces computational demands by enabling an early exit strategy. This feature opens the door to including a human-in-the-loop in LLM's decision-making process, enhancing interaction and intervention possibilities.

Our innovation represents a method of decoupling training and inference, a concept previously explored through strategies like CoT. By integrating KV cache management with specific prompting strategies, our approach serves as an orthogonal complement to existing purely prompt-based techniques. We hope to open a new line of research on KV cache aware prompting strategies. These strategies have the potential to enhance the reasoning capabilities of LLMs while also introducing a layer of interpretability.

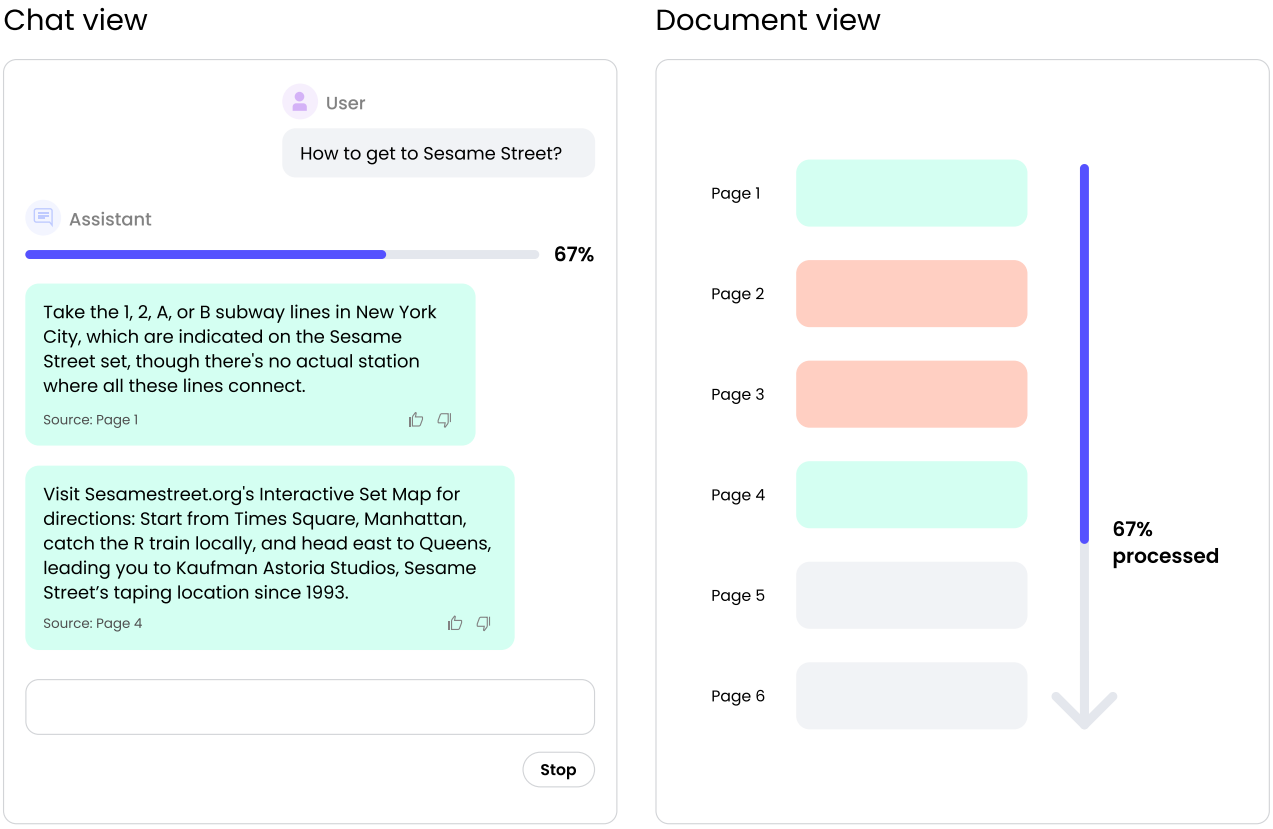

Figure 4: **WiM interactive retrieval design.** On the right, the document view displays the progress of processed segments, which can also be labeled based on the relevance identified by the LLM classifier. On the left, the chat view includes a progress bar that reflects the processing of segments. Here, users can interact with the streamed margins by giving a thumbs up or down, and these interactions are considered in the final response. Each margin corresponds to a specific document segment.

8 Future Directions

We see numerous unexplored opportunities to expand on our findings in future research:

- The efficiency and effectiveness of WiM inference pattern can be improved by focusing on both optimizing computational cost through better KV cache management and improving baseline models' performance by finetuning models specifically for the extraction and classification tasks. A further improvement could be also determining the optimal segment size (which can be different for each model).
- We imagine our inference pattern to be a good choice for transformers performing attention in segmented context windows, instead of accessing the whole sequence. Examples include Transformer XL [6] and transformers incorporating Infini-Attention [45]. The latter attempts to compress past key values into Fast Weight Memory [14]. We hypthesize that WiM could lower the requirements for the compressive memory that would represented only a limited past context whereas the (relevant) long term memory would be stored as WiM's margins.

9 Related Work

External Memory and Retrieval Methods Memory augmentation in Large Language Models (LLMs) involves integrating external memory banks, such as k-nearest neighbor (k-NN) models, to use textual similarities for generating context-aware completions [9]. These k-NN based LLMs excel in managing irregular patterns and factual data [1]. Additionally, approaches like Retrieval-Augmented Generation (RAG) [12] and "Entities as Experts" [7] link LLMs with external data sources—ranging from structured knowledge graphs [18] to learned entity embeddings. Such methods allow LLMs to access and utilize external information to enhance response accuracy and relevance.

Scratchpad Mechanisms A method for intermediate computation in LLMs involves the use of "scratchpads" or CoT [34] as a method for improving handling of sustained multi-step computations. Adopted from the findings of "Show Your Work: Scratchpads for Intermediate Computation with Lan-

guage Models" [13] this method enables LLMs to show their logic step-by-step, similar to a human using paper to jot down interim calculations. By training Transformers to sequentially output the results of intermediate steps rather than only final answers, LLMs demonstrate enhanced performance on complex tasks that go beyond single-step reasoning, such as long addition and program execution. This method not only helps the model maintain and extend context dynamically but also aids in debugging and understanding model decisions [10]. Further studies into length generalization [16] have demonstrated that traditional fine-tuning techniques on tasks requiring such generalizations often encounter significant limitations. By integrating scratchpad-like methodologies, these language models can achieve a notable improvement in handling progressively longer text spans. This enhancement proves particularly valuable for challenges such as theorem proving and extensive text synthesis. Here, the in-context learning combined with the sequential output of computed steps substantially bolsters task accuracy and model robustness [11, 15].

Context Aggregation Efficiency in context aggregation for LLMs have evolved with methods like Fusion-in-Decoder (FiD) and Map Reduce. FiD, used in models such as T5 and BART, consolidates contextual embeddings via encoder and decoder components to ensure comprehensive information integration [21, 8]. Conversely, LangChain's Map Reduce processes segments in parallel to quickly synthesize responses into a refined final output [17]. Parallel Context Windows (PCW) and Naive Bayes Context Extension (NBCE) further enhance handling of extended contexts by partitioning these into smaller segments for efficient parallel processing, optimizing both processing speed and response relevance [29, 19].

References

[1] Walter Daelemans, Antal van den Bosch, and Jakub Zavrel. *Forgetting Exceptions is Harmful in Language Learning*. 1998. arXiv: cs/9812021 [cs.CL].

[2] Steven Bird, Ewan Klein, and Edward Loper. *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc.", 2009.

[3] Pranav Rajpurkar, Robin Jia, and Percy Liang. *Know What You Don't Know: Unanswerable Questions for SQuAD*. 2018. arXiv: 1806.03822 [cs.CL]. URL: https://arxiv.org/abs/1806.03822.

[4] Zhilin Yang et al. *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. 2018. arXiv: 1809.09600 [cs.CL]. URL: https://arxiv.org/abs/1809.09600.

[5] Zihang Dai et al. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. 2019. arXiv: 1901.02860 [cs.LG]. URL: https://arxiv.org/abs/1901.02860.

[6] Zihang Dai et al. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. 2019. arXiv: 1901.02860 [cs.LG].

[7] Thibault Févry et al. *Entities as Experts: Sparse Memory Access with Entity Supervision*. 2020. arXiv: 2004.07202 [cs.CL].

[8] Gautier Izacard and Edouard Grave. "Leveraging passage retrieval with generative models for open domain question answering". In: *arXiv preprint arXiv:2007.01282* (2020).

[9] Urvashi Khandelwal et al. *Generalization through Memorization: Nearest Neighbor Language Models*. 2020. arXiv: 1911.00172 [cs.CL].

[10] Jacob Austin et al. *Program Synthesis with Large Language Models*. 2021. arXiv: 2108.07732 [cs.PL].

[11] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG].

[12] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL].

[13] Maxwell Nye et al. *Show Your Work: Scratchpads for Intermediate Computation with Language Models*. 2021. arXiv: 2112.00114 [cs.LG].

[14] Imanol Schlag, Tsendsuren Munkhdalai, and Jürgen Schmidhuber. *Learning Associative Inference Using Fast Weight Memory*. 2021. arXiv: 2011.07831 [cs.LG]. URL: https://arxiv.org/abs/2011.07831.

[15] Yuhuai Wu et al. *INT: An Inequality Benchmark for Evaluating Generalization in Theorem Proving*. 2021. arXiv: 2007.02924 [cs.AI].

[16] Cem Anil et al. *Exploring Length Generalization in Large Language Models*. 2022. arXiv: 2207.04901 [cs.CL].

[17] Harrison Chase. *LangChain*. https://github.com/langchain-ai/langchain. Oct. 2022.

[18] Qi Liu, Dani Yogatama, and Phil Blunsom. *Relational Memory Augmented Language Models*. 2022. arXiv: 2201.09680 [cs.CL].

[19] Nir Ratner et al. "Parallel context windows improve in-context learning of large language models". In: *arXiv preprint arXiv:2212.10947* (2022).

[20] Amey Agrawal et al. *SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills*. 2023. arXiv: 2308.16369 [cs.LG]. URL: https://arxiv.org/abs/2308.16369.

[21] Maor Ivgi, Uri Shaham, and Jonathan Berant. "Efficient long-text understanding with short-text models". In: *Transactions of the Association for Computational Linguistics* 11 (2023), pp. 284–299.

[22] Woosuk Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG]. URL: https://arxiv.org/abs/2309.06180.

[23] Jiaqi Li et al. *LooGLE: Can Long-Context Language Models Understand Long Contexts?* 2023. arXiv: 2311.04939 [cs.CL]. URL: https://arxiv.org/abs/2311.04939.

[24] Nelson F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. 2023. arXiv: 2307.03172 [cs.CL]. URL: https://arxiv.org/abs/2307.03172.

[25] Amirkeivan Mohtashami and Martin Jaggi. *Landmark Attention: Random-Access Infinite Context Length for Transformers*. 2023. arXiv: 2305.16300 [cs.CL]. URL: https://arxiv.org/abs/2305.16300.

[26] OpenAI. *GPT 4*. 2023. URL: https://openai.com/research/gpt-4.

[27] Bowen Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models*. 2023. arXiv: 2309.00071 [cs.CL]. URL: https://arxiv.org/abs/2309.00071.

[28] Uri Shaham et al. *ZeroSCROLLS: A Zero-Shot Benchmark for Long Text Understanding*. 2023. arXiv: 2305.14196 [cs.CL]. URL: https://arxiv.org/abs/2305.14196.

[29] Jianlin Su. *NBCE: Handling Length in Context Expansion of LLM with Naive Bayes*. https://spaces.ac.cn/archives/9617. 2023.

[30] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL]. URL: https://arxiv.org/abs/2104.09864.

[31] Szymon Tworkowski et al. *Focused Transformer: Contrastive Training for Context Scaling*. 2023. arXiv: 2307.03170 [cs.CL].

[32] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762.

[33] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

[34] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL].

[35] Shunyu Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. 2023. arXiv: 2305.10601 [cs.CL]. URL: https://arxiv.org/abs/2305.10601.

[36] Hattie Zhou et al. *What Algorithms can Transformers Learn? A Study in Length Generalization*. 2023. arXiv: 2310.16028 [cs.LG]. URL: https://arxiv.org/abs/2310.16028.

[37] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 2024. arXiv: 2404.14219 [cs.CL]. URL: https://arxiv.org/abs/2404.14219.

[38] Yushi Bai et al. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*. 2024. arXiv: 2308.14508 [cs.CL]. URL: https://arxiv.org/abs/2308.14508.

[39] Maciej Besta et al. "Graph of Thoughts: Solving Elaborate Problems with Large Language Models". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.16 (Mar. 2024), 17682–17690. ISSN: 2159-5399. DOI: 10.1609/aaai.v38i16.29720. URL: http://dx.doi.org/10.1609/aaai.v38i16.29720.

[40] Yukang Chen et al. *LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models*. 2024. arXiv: 2309.12307 [cs.CL]. URL: https://arxiv.org/abs/2309.12307.

[41] Abhimanyu Dubey et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[42] Tao Ge et al. *In-context Autoencoder for Context Compression in a Large Language Model*. 2024. arXiv: 2307.06945 [cs.CL]. URL: https://arxiv.org/abs/2307.06945.

[43] Cheng-Ping Hsieh et al. *RULER: What's the Real Context Size of Your Long-Context Language Models?* 2024. arXiv: 2404.06654 [cs.CL]. URL: https://arxiv.org/abs/2404.06654.

[44] Jesse Mu, Xiang Lisa Li, and Noah Goodman. *Learning to Compress Prompts with Gist Tokens*. 2024. arXiv: 2304.08467 [cs.CL]. URL: https://arxiv.org/abs/2304.08467.

[45] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. *Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention*. 2024. arXiv: 2404.07143 [cs.CL]. URL: https://arxiv.org/abs/2404.07143.

[46] Yixuan Tang and Yi Yang. *MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries*. 2024. arXiv: 2401.15391 [cs.CL]. URL: https://arxiv.org/abs/2401.15391.

[47] vLLM. *Performance and Tuning*. 2024. URL: https://docs.vllm.ai/en/latest/models/performance.html#chunked-prefill.

[48] An Yang et al. *Qwen2 Technical Report*. 2024. arXiv: 2407.10671 [cs.CL]. URL: https://arxiv.org/abs/2407.10671.

A Appendix - Decoupling Extraction and Classification

Writing in the margins generates supplemental information by leveraging a partially prefilled KV cache. Each subsequent segment c in the KV cache can be used to generate an annotation, known as a "margin note". To avoid providing the model with all the margins, we ask the model to generate the first token corresponding to the margin classes: relevant vs. irrelevant. In this section, we explore the possibility of decoupling the extraction and classification steps, which will allow for using separate prompting strategies. This separation might further boost the performance of the WiM pattern. We demonstrate that one can use the same instance of the model to perform both the computation of the margins and their classification.

In a naive implementation of such overlapped computation, the user may treat the classification request as an additional sequence and batch it with the prefilling request; this approach would require a very large number of padding tokens to align the two sequences. A more computationally efficient solution is to pack the classification request into the same sequence used to prefill the context and adjust the attention mask accordingly. An example of such a mask is provided in Figure 5. This technique is utilized during the pre-training of language models to reduce the number of padding tokens.

	Hello	my	name	is	John	This	is	a	dog
Hello	1	0	0	0	0	0	0	0	0
my	1	1	0	0	0	0	0	0	0
name	1	1	1	0	0	0	0	0	0
is	1	1	1	1	0	0	0	0	0
John	1	1	1	1	1	0	0	0	0
This	0	0	0	0	0	1	0	0	0
is	0	0	0	0	0	1	1	0	0
a	0	0	0	0	0	1	1	1	0
dog	0	0	0	0	0	1	1	1	1

Figure 5: **Sequence packing.** Sequence packing allows to pack multiple unrelated documents in the same sequence. By adjusting the attention mask, we can avoid cross-contamination. This speeds up training time by reducing the number of padding tokens. A similar technique can also be used to inference from multiple prompts using the same sequence.

The first request to the language model would only contain the first segment c_1 and the additional extractive instruction I_A (the"extractive summary prompt"). The attention mask at this point is provided in Figure 6 and Figure 7. This would generate the first margin M_0. After generating M_0, the instruction prompt I_A and all the subsequent tokens generated in M_0 can be removed from the KV cache, leaving the KV cache only with c_1. In order to not grow or shrink a dynamically allocated KV cache, it is possible to use a static KV cache, as the number of total tokens in each segment, extractive instruction and classification prompt is known in advance, so is the maximum number of tokens for each margin M_i and classification result $\omega(I(M_i))$.

Having generated the first margin M_0, it is possible to add the second segment c_2 to generate the second margin M_1 while at the same time classifying the previously generated margin M_0. To do so, the KV cache is prefilled with subsequent tokens c_2, the extractive instruction I_A and a number of padding tokens to accommodate the generated tokens of margin M_1. Moreover, the KV cache is also expanded by adding the classification instruction $I(M_0)$ and a number of padding tokens to accommodate the generated tokens for the classification result $\omega(I(M_0))$. The attention mask at this point is provided in Figure 8.

Autoregressive token generation of the margin M_1 and the classification result $\omega(I(M_0))$ can be done in parallel by projecting the last token of each sub-sequence into logits. Each generated token can then be added in place of a padding token in each subsequence to generate successive tokens. Token generation at this stage is shown in Figure 9.

		c_1				I_A		PAD		
		K1	K2	K3	K4	K5	K6	P	P	...
c_1	Q1	1	0	0	0	0	0	0	0	0
	Q2	1	1	0	0	0	0	0	0	0
	Q3	1	1	1	0	0	0	0	0	0
	Q4	1	1	1	1	0	0	0	0	0
I_A	Q5	1	1	1	1	1	0	0	0	0
	Q6	1	1	1	1	1	1	0	0	0

Figure 6: **Prefilling of the first segment c_1 along with the extractive instruction I_A.** Padding tokens are shown for clarity in case of a statically allocated KV cache, but they do not needed to be attended to or used in the KV sequence when calculating the attention. The KV sequence should be a *slice* of the KV tensor that includes only non-padding tokens.

		c_1				I_A		M_0	PAD	
		K1	K2	K3	K4	K5	K6	K7	P	...
M_0	Q7	1	1	1	1	1	1	1	0	0

Figure 7: **Token generation using the prefilled KV cache.** Each generated token replaces a padding token in the KV cache.

By using a statically allocated KV cache and by keeping track of how many tokens are used in it, it is possible to use a partial *view* (also known as "tensor slicing") of the KV tensor without any computational overhead. It is also possible to use techniques like PagedAttention [22] to allocate the KV cache block by block, in order to optimize the memory consumption while benefiting from a partial static allocation.

		c_1				c_2				I_A		PAD		$I(M_0)$			PAD		
		K1	K2	K3	K4	K5	K6	K7	K8	K9	K10	P	...	K21	K22	K23	P	P	...
c_2	Q5	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	Q6	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
	Q7	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
	Q8	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0
I_A	Q9	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
	Q10	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
$I(M_0)$	Q21	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
	Q22	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
	Q23	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0

Figure 8: **Prefilling of the second segment c_1 along with the extractive instruction I_A.** In this case the padding tokens between I_A and $I(M_0)$ must be included in the KV sequence when calculating the attention to retain the memory continuity of the tensor, but the terminal padding tokens need not to. Each token in the second segment c_2 needs to attend all tokens in the first segment c_1. The classification prompt $I(M_0)$ be considered a completely separate document in the same sequence as prefilling.

		c_1				c_2				I_A		M_1	PAD	$I(M_0)$			$\omega(I(M_0))$	PAD	
		K1	K2	K3	K4	K5	K6	K7	K8	K9	K10	K11	...	K21	K22	K23	K24	P	...
M_1	Q11	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0
$\omega(I(M_0))$	Q24	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	0

Figure 9: **Parallel token generation of the margin M_1 and the classification result $\omega(I(M_0))$.** Each generated token replaces a padding token in its specific subsequence.

B Appendix - Prompts

B.1 Evaluation

B.1.1 Prompt used with accuracy metric for SQuAD, HtopotQA and MultiHop-RAG

```
Evaluate the following exam answer. I will provide you with the query, target answer(s) and the answer
    ↪ provided by the student.
The student's answer does not need to preserve the casing of the target answers, and slight variations in
    ↪ phrasing are acceptable, provided the meaning remains correct.
Provide the answer in the format: <YES/NO>#<Explanation>.

Here are the rules:
- If the student's answer is correct - start your answer with YES#
- If the student's answer is wrong or it is missing - start your answer with NO#

Example answers:

QUERY: As of 2016, about what percentage of adults aged 18 years or older were overweight?
TARGET: 40%, forty percent
ANSWER: forty percent
YES#The student's answer is correct.

QUERY: What is the value of p in 24 = 2p?
TARGET: 12, 12.0
ANSWER: five
NO#The student's answer is wrong.

QUERY: What is the 'Lotus principle'?
TARGET: The so-called Lotus principle is that 'restrictions upon the independence of States cannot
    ↪ therefore be presumed
```

```
ANSWER: The Lotus principle is a horticultural technique developed in ancient Egypt for cross-pollinating
    ↪ lotus flowers with roses to create fragrant, floating gardens.
NO#No, the student's explanation is wrong.

QUERY: {query}
TARGET: {target}
ANSWER: {answer}
```

B.1.2 Prompt used with F1 metric for CWE

```
Evaluate the following exam answer. The student's task was to identify the most common words in text. I
    ↪ will provide you with the target answer(s) - 10 words - and the student's response.
The student's answer does not need to preserve the casing of the target words.
Provide your assessment in the format: <Positive>#<Negative>#<Explanation>.

Here are the rules:
- <Positive> denotes the number of common words that the student has correctly identified (i.e., those
    ↪ included in the target list).
- <Negative> represents the number of words the student has identified as common but are incorrect (i.e.,
    ↪ not included in the target list).
- If the student's answer is non-relevant, simply return: 0#10#The answer is garbage.
- If the student's answer includes Python code, mention "python" in your explanation.
- If the student provides the full answer twice, consider only the first submission.

Example answers:

TARGET: jealousy, access, chase, recess, head, animal, capon, hops, unequaled, mozzarella
ANSWER: Based on the provided content and the margin notes, the 10 most common words in the input are:\n\n1
    ↪ . chase (100 occurrences)\n2. unequaled (99 occurrences)\n3. access (98 occurrences)\n4. hops (97
    ↪ occurrences)\n5. animal (95 occurrences)\n6. jealousy (96 occurrences)\n7. mozzarella (94
    ↪ occurrences)\n8. recess (93 occurrences)\n9. capon (91 occurrences)\n10. oxygen (91 occurrences),
9#1#The student's answer contains 9 correct words out of 10 given.

TARGET: seeder, workout, wafer, chronograph, liability, hovercraft, nestling, alloy, netball, addicted
ANSWER: To find the 10 most common words in the given input, we need to count the occurrences of each word
    ↪ and then sort them in descending order. Here are the 10 most common words and their counts:\n\n1. \"
    ↪ floodplain\" - 25 occurrences\n2. \"acetate\" - 24 occurrences\n3. \"nondisclosure\" - 23
    ↪ occurrences\n4. \"caterpillar\" - 22 occurrences\n5. \"moai\" - 21 occurrences\n6. \"video\" - 20
    ↪ occurrences\n7. \"bacterium\" - 19 occurrences\n8. \"frog\" - 19 occurrences\n9. \"carnivore\" - 18
    ↪ occurrences\n10. \"cattle\" - 18 occurrences\n\nNote: The input provided is a long list of words,
    ↪ and the above list is based on the most frequent words found in the list. The actual list may vary
    ↪ depending on the specific input.
0#10#The student's answer provides only a generic answer and fails to solve the problem.

TARGET: jealousy, access, chase, recess, head, animal, capon, hops, unequaled, mozzarella
ANSWER: To find the 10 most common words in the provided input, we can use a Python script that reads the
    ↪ text, splits it into words, counts the occurrences of each word, and then prints the top 10 most
    ↪ common words along with their counts. Here's a Python script that accomplishes this:\n\npython\nfrom
    ↪  collections import Counter\n\n# Read the text from the input\nwith open('input.txt', 'r') as file:\
    ↪ n text = file.read()\n\n# Split the text into words\nwords = text.split()\n\n# Count the occurrences
    ↪  of each word\nword_counts = Counter(words)\n\n# Get the 10 most common words\nmost_common_words =
    ↪ word_counts.most_common(10)\n\n# Print the 10 most common words and their counts\nfor word, count in
    ↪  most_common_words:\n print(word, count) The provided text\ntext = 1.jealousy 2. gauge 3. work 4.
    ↪ townhouse 5. ubiquitous 6. regulator 7. oxygen 8. verdict 9. war 10. verdict 11. rag 12. rag
1#11#The student's answer contains python code. One word is correct but it contains also other 11 incorrect
    ↪  words.

TARGET: {target}
ANSWER: {answer}
```